

PHP 8 CONEXÃO COM MYSQL



PROF. SERGIO LUIZ DA SILVEIRA

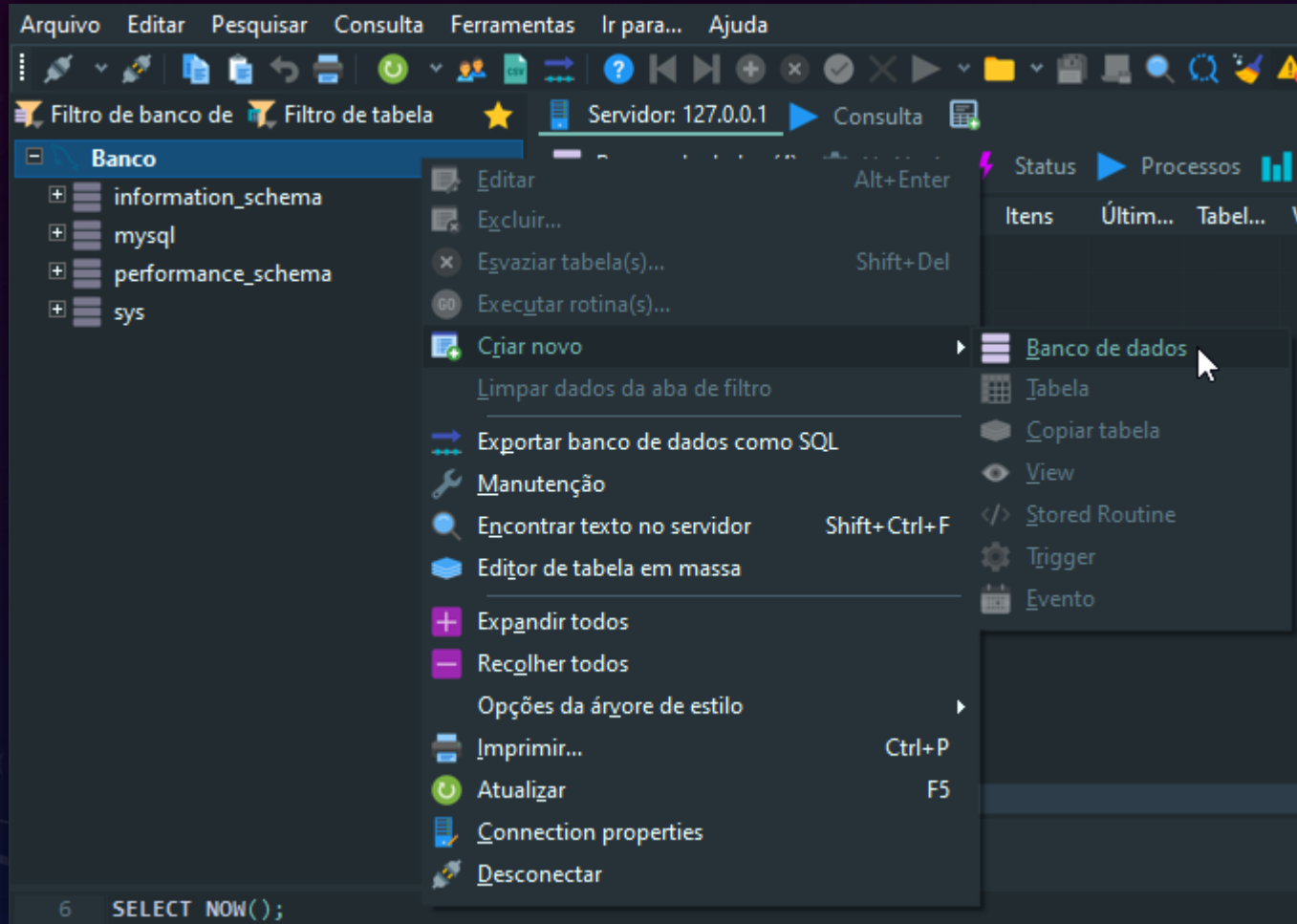
FORMAS DE CONEXÃO E ACESSO

- mysqli (MySQL Improved)
- PDO (PHP Data Objects)

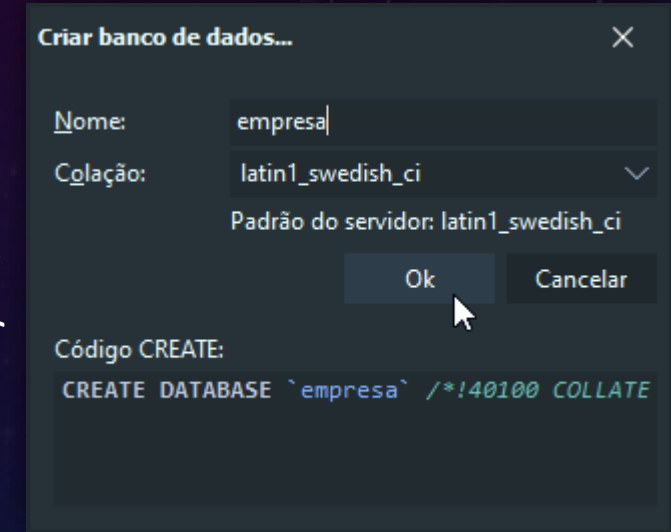


BANCO DE DADOS EXEMPLO

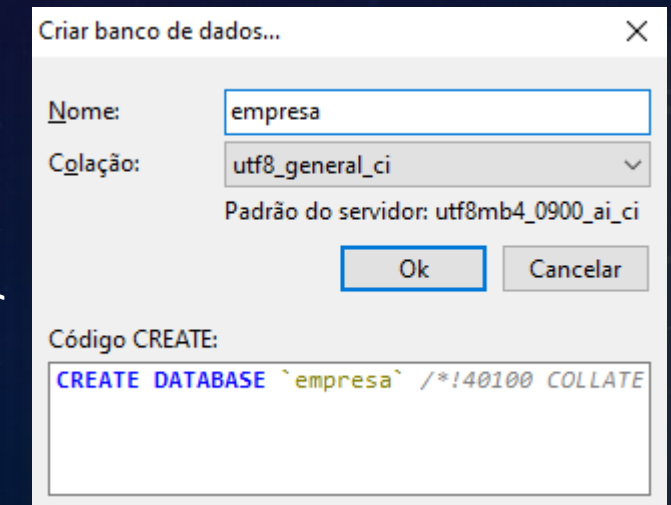
- Banco: empresa



MySQL 5.7

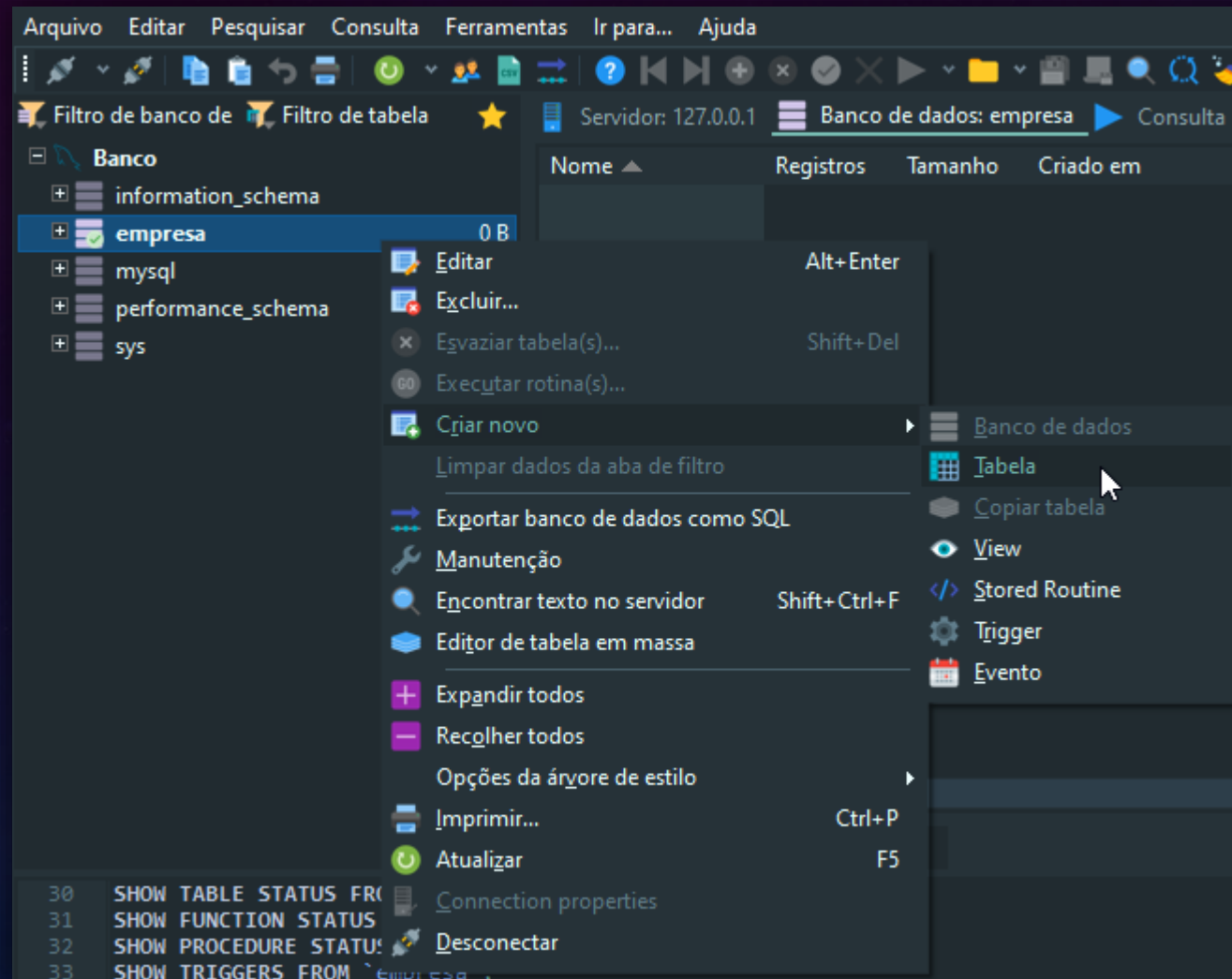


MySQL 8



O BANCO DE DADOS

- Tabela: cliente



O BANCO DE DADOS

- Tabela: cliente

Servidor: 127.0.0.1 Banco de dados: empresa Tabela: [Untitled] Consulta

Básico Opções Indexes Chaves-estrangeiras Partições Código CREATE

Nome: cliente

Comentário:

Colunas: + Adicionar x Remover ▲ Mover para cima ▼ Mover para baixo

Nome	Tipo de dados	Tamanho/It...	Unsign...	Permitir...	Zerofill	Padrão	Co
id_cliente	INT		☒	☒	☒	☐ Nenhum valor padrão	
nome	VARCHAR	50	☐	☒	☐	☐ Custom text:	
endereco	VARCHAR	80	☐	☒	☐		
telefone	VARCHAR	20	☐	☒	☐		
email	VARCHAR	50	☐	☒	☐	☐ NULL	
						☐ Expressão:	

Ajuda Descartar Salvar

Filtro: Expressão regular

, EVENT_NAME AS `Name` FROM information_schema.`EVENTS` WHERE `EVENT\_SCHEMA`

dress 000000000040EBDC in module 'heidisql.exe'. Read of address 0000000007.

On update:

☒ AUTO_INCREMENT

Ok Cancelar

Colunas: + Adicionar x Remover ▲ Mover para cima ▼ Mover para baixo

#	Nome	Tipo de dados	Tamanho/It...	Unsign...	Permitir...
1	id_cliente	INT		☒	☒
2	nome			☐	☒
3	endereco			☐	☒
4	telefone			☐	☒
5	email			☐	☒

Ajuda De

Filtro: Expressão

EVENT_NAME AS `N

ress 000000000040EBDC in module 'heidisql.exe'. Re

Crj ar novo index

Adicionar ao index

PRIMARY

KEY

UNIQUE

FULLTEXT

POSSIBILIDADES

- SELECT
 - WHERE (filtro)
 - JOIN (juntando tabelas)
 - GROUP BY (agrupando)
- INSERT
- UPDATE
- DELETE

```
SELECT * FROM users  
WHERE username = 'teo'
```

```
SELECT * FROM users  
ORDER BY username
```

```
INSERT INTO  
  users(username, name)  
VALUES ('joe', 'Joe Li');
```

```
UPDATE posts  
SET title = New Title'  
WHERE id = 2;
```

```
DELETE FROM posts  
WHERE id = 6;
```

MYSQLI - CONEXÃO

- Por procedimento

- `$conexao = mysqli_connect ('localhost','usuário','senha','bancodedados');`

- Por orientação a objeto (recomendado)

- `$conexão = new mysqli ('localhost','usuário','senha','bancodedados');`

CONEXÃO COM VERIFICAÇÃO PROCEDURAL

```
<?php
```

```
// Definição das credenciais de acesso ao banco de dados
```

```
$nomeservidor = "localhost";           // Endereço do servidor MySQL
```

```
$usuario = "root";                     // Nome de usuário do banco
```

```
$senha = "";                           // Senha do banco
```

```
$bancodedados = "empresa";           // Nome do banco de dados
```

```
// Criação da conexão com MySQLi
```

```
$conn = mysqli_connect($nomeservidor, $usuario, $senha,  
$bancodedados);
```

CONTINUA...

CONEXÃO COM VERIFICAÇÃO PROCEDURAL

...CONTINUAÇÃO

// Verificação da conexão

**if (!\$conn) { // Caso a conexão falhe, interrompe o script e
exibe uma mensagem de erro**

die("Conexão falhou: " . mysqli_connect_error());}

**// Configuração do conjunto de caracteres para evitar problemas
de acentuação**

mysqli_set_charset(\$conn, "utf8mb4");

CONTINUA...

CONEXÃO COM VERIFICAÇÃO PROCEDURAL

...CONTINUAÇÃO

```
// Mensagem indicando que a conexão foi estabelecida  
corretamente  
echo "Conexão bem-sucedida!";
```

COMO USAR - PROCEDURAL

// Consulta SQL para obter clientes

```
$sql = "SELECT id_cliente, nome, email FROM cliente";
```

```
$result = mysqli_query($conn, $sql);
```

// Verifica se há resultados na consulta

```
if (mysqli_num_rows($result) > 0) {
```

// Itera sobre os resultados e exibe os dados

```
    while ($linha = mysqli_fetch_assoc($result)) {  
        echo "ID: " . $linha["id_cliente"] . " - Nome: " . $linha["nome"] . " - Email: "  
        . $linha["email"] . "<br/>";    }  
}
```

```
else {
```

```
    echo "Nenhum resultado encontrado.";}  

```

// Fecha a conexão com o banco de dados

```
mysqli_close($conn);
```

```
?>
```

FIM!

COMO USAR - PROCEDURAL

```
// Consulta SQL para obter clientes
```

```
$sql = "SELECT id_cliente, nome, email FROM cliente";
```

```
$result = mysqli_query($conn, $sql);
```

```
// Verifica se há resultados na consulta
```

```
if (mysqli_num_rows($result) > 0) {
```

```
// Itera sobre os resultados
```

```
while ($linha =
```

```
echo "
```

```
. $linha
```

```
} else {
```

```
echo
```

```
// Fecha a conexão com o banco de dados
```

```
mysqli_close($conn);
```

```
?>
```

Uso de `mysqli_fetch_assoc()`, melhora a eficiência ao evitar a repetição de índice numérico.

```
$linha["nome"] . " - Email: "
```


Conexão usando MySQLi em POO

Arquivo para Conexão ao Banco de Dados

Arquivo `conexao.php`

```
1. <?php
2. // Habilita relatório detalhado de erros no MySQLi
3. mysqli_report(MYSQLI_REPORT_ERROR |
   MYSQLI_REPORT_STRICT);
```

Arquivo `conexao.php`

```
/**
 * Função para conectar ao banco de dados
 * Retorna um objeto de conexão MySQLi ou interrompe o script em caso de
 * erro.
 */
function conectadb() {
    // Configuração do banco de dados
    $endereco = "localhost"; // Endereço do servidor MySQL
    $usuario = "root";       // Nome de usuário do banco de dados
    $senha = "";             // Senha do banco de dados
    $banco = "empresa";      // Nome do banco de dados
}
```


Arquivo conexao.php

```
try {  
    // Criação da conexão  
    $con = new mysqli($endereco, $usuario, $senha, $banco);  
  
    // Definição do conjunto de caracteres para evitar problemas de  
    acentuação  
    $con->set_charset("utf8mb4");      // Retorna o objeto de conexão  
    return $con;  
} catch (Exception $e) {  
    // Exibe uma mensagem de erro e encerra o script  
    die("Erro na conexão: " . $e->getMessage());  
}  
}
```

?>

Arquivo para Consulta **no Banco de Dados**

Arquivo select_cliente.php

```
<?php
```

```
// Inclui o arquivo de conexão com o banco de dados
```

```
require_once "conexao.php";
```

```
// Estabelece conexão
```

```
$conexao = conectadb();
```

```
// Define a consulta SQL para buscar clientes
```

```
$sql = "SELECT id_cliente, nome, email FROM cliente";
```

```
// Executa a consulta no banco
```

```
$result = $conexao->query($sql);
```

Arquivo select_cliente.php

```
// Verifica se há registros retornados
if ($result->num_rows > 0) {
    // Itera sobre os resultados e exibe cada cliente encontrado
    while ($linha = $result->fetch_assoc()) {
        echo "ID: " . $linha["id_cliente"] . " - Nome: " .
$linha["nome"] . " - Email: " . $linha["email"] . "<br/>";
    }
} else {
    // Caso nenhum resultado seja encontrado
    echo "Nenhum cliente cadastrado.";}
// Fecha a conexão
$conexao->close();
```

?>

Arquivo para GRAVAR no Banco de Dados

com passagem de valores

Arquivo insert_cliente.php

```
<?php
```

```
// Inclui o arquivo de conexão com o banco de dados  
require_once "conexao.php";
```

```
// Estabelece conexão  
$conexao = conectadb();
```

```
// Definição dos valores para inserção  
$nome = "João da Silva";  
$endereco = "Rua Kalamango, 32";  
$telefone = "(41) 5555-5555";  
$email = "joaosilva@teste.com";
```

Arquivo insert_cliente.php

// Prepara a consulta SQL usando `prepare()` para evitar SQL Injection

```
$stmt = $conexao->prepare("INSERT INTO cliente  
(nome, endereco, telefone, email) VALUES (?, ?, ?, ?)");
```

// Associa os parâmetros aos valores da consulta

```
$stmt->bind_param("ssss", $nome, $endereco,  
$telefone, $email);
```

Arquivo insert_cliente.php

// Executa a inserção

```
if ($stmt->execute()) {  
    echo "Cliente adicionado com sucesso!";  
} else {  
    echo "Erro ao adicionar cliente: " . $stmt->error;  
}
```

// Fecha a consulta e a conexão com o banco de dados

```
$stmt->close();  
$conexao->close();?>
```

Arquivo para ALTERAR no Banco de Dados

com passagem de valores

Arquivo update_cliente.php

```
<?php
```

```
// Inclui o arquivo de conexão com o banco de dados  
require_once "conexao.php";
```

```
// Estabelece conexão  
$conexao = conectadb();
```

Arquivo update_cliente.php

// Define os novos valores

\$nome = "Maria da Silva";

\$endereco = "Rua Kalamango, 32";

\$telefone = "(41) 5555-5555";

\$email = "mariasilva@teste.com";

// Define o ID do cliente a ser atualizado

\$id_cliente = 1;

Arquivo update_cliente.php

// Prepara a consulta SQL segura

```
$stmt = $conexao->prepare("UPDATE cliente SET nome  
= ?, endereco = ?, telefone = ?, email = ? WHERE  
id_cliente = ?");
```

// Associa os parâmetros aos valores da consulta

```
$stmt->bind_param("ssssi", $nome, $endereco,  
$telefone, $email, $id_cliente);
```

Arquivo update_cliente.php

```
// Executa a atualização
```

```
if ($stmt->execute()) {
```

```
    echo "Cliente atualizado com sucesso!";
```

```
} else {
```

```
    echo "Erro ao atualizar cliente: " . $stmt->error;}
```

```
// Fecha a consulta e a conexão
```

```
$stmt->close();
```

```
$conexao->close();?>
```

Arquivo para **APAGAR** no Banco de Dados

com passagem de valores

Arquivo delete_cliente.php

```
<?php
```

```
// Inclui o arquivo de conexão com o banco de dados  
require_once "conexao.php";
```

```
// Estabelece conexão  
$conexao = conectadb();
```

```
// Define o ID do cliente que será excluído  
$id_cliente = 1;
```

Arquivo delete_cliente.php

// Prepara a consulta SQL segura

```
$stmt = $conexao->prepare("DELETE FROM cliente  
WHERE id_cliente = ?");
```

// Associa o parâmetro ao valor da consulta

```
$stmt->bind_param("i", $id_cliente);
```

Arquivo delete_cliente.php

```
// Executa a exclusão
if ($stmt->execute()) {
    echo "Cliente removido com sucesso!";
} else {
    echo "Erro ao remover cliente: " . $stmt->error;
}
```

```
// Fecha a consulta e a conexão
```

```
$stmt->close();
$conexao->close();
```

```
?>
```

Organização para o projeto PDO

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

Durante a aula, iremos desenvolver **um sistema completo** para gerenciar clientes usando **PHP** e **PDO**, seguindo boas práticas de modularização.

Nosso projeto será composto pelos seguintes arquivos:

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

1. **conexao.php** → Responsável pela conexão com o banco de dados.
 - Encapsula a lógica de conexão com o MySQL utilizando **PDO**.
 - Garante **tratamento de erros** para evitar problemas inesperados.
 - Configura **atributos de segurança**, como **ERRMODE_EXCEPTION**.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

2. funcoesBanco.php → Centraliza funções reutilizáveis para acesso ao banco.

- Aqui, criamos funções separadas para **inserir, atualizar, remover e listar clientes**.
- Melhora a **organização e reaproveitamento** do código.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

3. `inserirCliente.php` → Formulário para cadastrar novos clientes.

- Permite ao usuário inserir nome, endereço, telefone e e-mail.
- Envia os dados para `**`processarInsercao.php`**` via **`POST`**.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

4. **processarInsercao.php** → **Processa a
inserção dos clientes no banco.**

- Recebe os dados do formulário e executa **INSERT** no banco usando **PDO**.
- Sanitiza as entradas para evitar **SQL Injection** e problemas de segurança.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

5. atualizarCliente.php → **Formulário para editar informações de clientes.**

- Permite que o usuário busque um cliente pelo ID e atualize seus dados.
- Envia as informações para **`processarAtualizacao.php`**.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

6. **processarAtualizacao.php** → **Executa a atualização dos dados no banco.**

Recebe os valores do formulário e realiza o **UPDATE** na tabela de clientes.

Implementa segurança contra **injeção de código malicioso**.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

7. **deletarCliente.php** → Formulário para excluir clientes.

- Solicita o **ID** do cliente que deve ser removido do banco.
- Envia a solicitação de exclusão para **`processarDelecao.php`**.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

8. **processarDelecao.php** → Executa a exclusão do cliente no banco de dados.
- Recebe o **ID** e realiza a **remoção segura** com **DELETE** via **PDO**.
 - Evita **erros e vulnerabilidades**, verificando se o ID informado é válido.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

9. `listarClientes.php` → Exibe os clientes cadastrados no sistema.

- Recupera os registros da tabela de clientes e os apresenta em uma ****tabela HTML****.
- Usa **`PDO`** para garantir segurança e organização na consulta.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

10. buscarCliente.php → Este arquivo é responsável por listar todos os clientes cadastrados no banco de dados.

- Consulta todos os clientes na base de dados, ordenados alfabeticamente.
- Exibe os dados em uma **tabela dinâmica**, permitindo visualização rápida.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

10. buscarCliente.php → Este arquivo é responsável por listar todos os clientes cadastrados no banco de dados.

- Oferece um botão **"Editar"**, que redireciona para **`atualizarCliente.php`**, permitindo a edição do cliente selecionado.
- Se não houver clientes cadastrados, exibe uma ****mensagem amigável** informando a ausência de registros.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

11. `pesquisarCliente.php` → Este arquivo permite que o usuário pesquise um cliente específico por **ID** ou **Nome**.

- Aceita **ID do cliente** para busca direta.
- Aceita **Nome do cliente**, trazendo todos os registros que correspondem ao termo pesquisado.
- Exibe os **resultados da pesquisa** em formato de tabela, permitindo ao usuário escolher qual cliente deseja editar.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

11. `pesquisarCliente.php` → Este arquivo permite que o usuário pesquise um cliente específico por **ID** ou **Nome**.

- Se não houver clientes correspondentes à busca, exibe uma **mensagem amigável** informando que nenhum registro foi encontrado.
- Redireciona para `**atualizarCliente.php**` ao clicar em "Editar", garantindo que o cliente correto seja modificado.

ARQUIVOS DO PROJETO: ESTRUTURA E FINALIDADE

11. `pesquisarCliente.php` → Este arquivo permite que o usuário pesquise um cliente específico por **ID** ou **Nome**.

Este arquivo é ideal para situações onde o usuário deseja **filtrar clientes** antes de editar um cadastro.



OBJETIVO DA ESTRUTURA

Com essa divisão de arquivos, conseguimos:

- ✓ **Organizar melhor o código**, separando responsabilidades de cada função.

- ✓ **Reaproveitar funções**, centralizando a lógica de banco em `funcoesBanco.php`.

- ✓ **Garantir segurança**, evitando falhas como **SQL Injection**.

- ✓ **Facilitar a manutenção**, permitindo futuras melhorias sem desorganizar o código.

Arquivo de CONEXAO **no Banco de Dados**

Arquivo conexao.php

```
function conectarBanco() {  
    $dsn = "mysql:host=localhost;dbname=empresa;charset=utf8";  
    $usuario = "root";  
    $senha = "";  
    try {  
        $conexao = new PDO($dsn, $usuario, $senha, [  
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC  
        ]);  
        return $conexao;  
    } catch (PDOException $e) {  
        error_log("Erro ao conectar ao banco: " . $e->getMessage());  
        // LOG SEM EXPOR ERRO AO USUÁRIO  
        die("Erro ao conectar ao banco.");  
    }  
}
```

Arquivo conexao.php

✓ Com essa abordagem, você pode incluir `require_once 'conexao.php';` nos outros arquivos quando precisar da conexão.

✓ Essa estrutura modular melhora a organização, evita repetições e torna o código mais fácil de manter. Me avise se quiser mais detalhes!

Arquivo conexao.php

Encapsulamento: Criamos uma função reutilizável para conexão.

Charset UTF-8: Previne problemas de codificação de caracteres

Configuração de ATTR_DEFAULT_FETCH_MODE: Evita a necessidade de fetch() manualmente, todas as consultas retornam arrays associativos.

Registro de erro com error_log(): Garante que informações sensíveis não apareçam para o usuário.

Arquivo conexao.php

```
function conectarBanco() {  
    $dsn = "mysql:host=localhost;dbname=empresa;charset=utf8";  
    $usuario = "root";  
    $senha = "";  
    try {  
        $conexao = new PDO($dsn, $usuario, $senha, [  
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC  
        ]);  
        return $conexao;  
    } catch (PDOException $e) {  
        error_log("Erro ao conectar ao banco: " . $e->getMessage());  
        // LOG SEM EXPOR ERRO AO USUÁRIO  
        die("Erro ao conectar ao banco.");  
    }  
}
```

PDO – ERRMODE

```
$conexao->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

PDO::ERRMODE_SILENT

Esse é o tipo padrão utilizado pelo PDO, basicamente o PDO seta internamente o código de um determinado erro, podendo ser resgatado através dos métodos `PDO::errorCode()` e `PDO::errorInfo()`.

PDO::ERRMODE_WARNING

Além de armazenar o código do erro, este tipo de manipulação de erro irá enviar uma mensagem `E_WARNING`, sendo este muito utilizado durante a depuração e/ou teste da aplicação.

PDO::ERRMODE_EXCEPTION

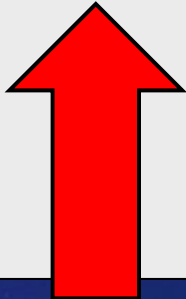
Além de armazenar o código de erro, este tipo de manipulação de erro irá lançar uma exceção `PDOException`, esta alternativa é recomendada, principalmente por deixar o código mais limpo e legível.



Arquivo de CONSULTAS **(PDO) no Banco de** **Dados**

Arquivo listarClientes.php

```
<?php  
require 'conexao.php';  
  
$conexao = conectarBanco();  
$stmt = $conexao->prepare("SELECT * FROM cliente");  
$stmt->execute();  
$clientes = $stmt->fetchAll();  
?>
```



Recupera dados do banco e
exibe em uma tabela HTML.

Arquivo listarClientes.php

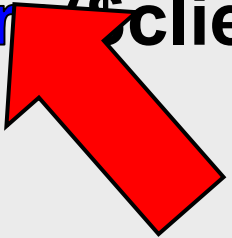
```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Lista de Clientes</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

Arquivo listarClientes.php

```
<h2>Lista de Clientes</h2>
<table border="1">
  <tr>
    <th>ID</th>
    <th>Nome</th>
    <th>Endereço</th>
    <th>Telefone</th>
    <th>E-mail</th>
  </tr>
```

Arquivo listarClientes.php

```
<?php foreach ($clientes as $cliente): ?>
    <tr>
        <td><?= htmlspecialchars($cliente["id_cliente"]) ?></td>
        <td><?= htmlspecialchars($cliente["nome"]) ?></td>
        <td><?= htmlspecialchars($cliente["endereco"]) ?></td>
        <td><?= htmlspecialchars($cliente["telefone"]) ?></td>
        <td><?= htmlspecialchars($cliente["email"]) ?></td>
    </tr>
<?php endforeach; ?>
</table>
</body>
</html>
```



- Usa **htmlspecialchars()** para evitar problemas de segurança ao exibir os dados.

Formulário para **GRAVAR** (PDO) no Banco de Dados

FRONT END

Arquivo `inserirCliente.php`

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Cadastro de Cliente</title>
  <link rel="stylesheet" href="style.css"> <!-- Arquivo opcional
para estilizar -->
</head>
```

Arquivo `inserirCliente.php`

```
<body>
  <h2>Cadastro de Cliente</h2>
  <form action="processarInsercao.php" method="POST">
    <label for="nome">Nome:</label>
    <input type="text" id="nome" name="nome" required>

    <label for="endereco">Endereço:</label>
    <input type="text" id="endereco" name="endereco"
required>

    <label for="telefone">Telefone:</label>
    <input type="text" id="telefone" name="telefone" required>
```

Arquivo `inserirCliente.php`

```
<label for="email">E-mail:</label>
```

```
<input type="email" id="email" name="email" required>
```

```
<button type="submit">Cadastrar Cliente</button>
```

```
</form>
```

```
</body>
```

```
</html>
```


Arquivo para GRAVAR (PDO) no Banco de Dados

BACK END

Arquivo processarInsercao.php

processarInsercao.php → Um script específico que **recebe dados do formulário** e **insere no banco**.

Ele age como uma **"ponte"** entre o HTML e o banco de dados.

Arquivo processarInsercao.php

```
<?php
require_once 'conexao.php';

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $conexao = conectarBanco();

    $sql = "INSERT INTO cliente (nome, endereco, telefone,
email)
        VALUES (:nome, :endereco, :telefone, :email)";
```

Arquivo processarInsercao.php

```
$stmt = $conexao->prepare($sql);  
$stmt->bindParam(":nome", $_POST["nome"]);  
$stmt->bindParam(":endereco", $_POST["endereco"]);  
$stmt->bindParam(":telefone", $_POST["telefone"]);  
$stmt->bindParam(":email", $_POST["email"]);  
try {  
    $stmt->execute();  
    echo "Cliente cadastrado com sucesso!";  
} catch (PDOException $e) {  
    error_log("Erro ao inserir cliente: " . $e->getMessage());  
    echo "Erro ao cadastrar cliente."  
}
```

```
}  
?>
```

Por que esse método é seguro?

Arquivo processarInsercao.php

```
$stmt = $conexao->prepare($sql);  
$stmt->bindParam(":nome", $_POST["nome"]);  
$stmt->bindParam(":endereco", $_POST["endereco"]);  
$stmt->bindParam(":telefone", $_POST["telefone"]);  
$stmt->bindParam(":email", $_POST["email"]);  
try {  
    $stmt->execute();  
}
```

- As variáveis são **tratadas separadamente** e **não são inseridas diretamente na consulta**.
- O banco de dados reconhece os **valores como parâmetros** e **não como comandos SQL**.

Arquivo processarInsercao.php



Conclusão: Sim, **processarInsercao.php** pode ser vulnerável se você inserir os dados diretamente na string SQL.

Mas ao usar **prepare()** e **bindParam()**, seu código se torna seguro contra SQL Injection.

Formulário para **ALTERAR (PDO)** no Banco de Dados

FRONT END

Arquivo atualizarCliente.php

```
<?php  
require_once 'conexao.php';  
  
$conexao = conectarBanco();  
  
// Obtendo o ID via GET  
$idCliente = $_GET['id'] ?? null;  
$cliente = null;  
$msgErro = '';
```


Arquivo atualizarCliente.php

```
// Função local para buscar cliente por ID
function buscarClientePorId($idCliente, $conexao) {
    $stmt = $conexao->prepare("SELECT id_cliente, nome,
endereco, telefone, email FROM cliente WHERE id_cliente =
:id");
    $stmt->bindParam(":id", $idCliente, PDO::PARAM_INT);
    $stmt->execute();
    return $stmt->fetch();
}
```

Arquivo atualizarCliente.php

```
// Se um ID foi enviado, busca o cliente no banco
if ($idCliente && is_numeric($idCliente)) {
    $cliente = buscarClientePorId($idCliente, $conexao);

    if (!$cliente) {
        $msgErro = "Erro: Cliente não encontrado.";
    }
} else {
    $msgErro = "Digite o ID do cliente para buscar os dados.";
}
?>
```

Arquivo atualizarCliente.php

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Atualizar Cliente</title>
  <link rel="stylesheet" href="style.css">
  <script>
    function habilitarEdicao(campo) {
document.getElementById(campo).removeAttribute("readonly");
    }
  </script>
</head>
<body>
```

Arquivo atualizarCliente.php

```
<h2>Atualizar Cliente</h2>
```

```
<!-- Se houver erro, exibe a mensagem e o campo de busca -->
```

```
<?php if ($msgErro): ?>
```

```
    <p style="color:red;"><?= htmlspecialchars($msgErro)  
?></p>
```

```
    <form action="atualizarCliente.php" method="GET">
```

```
        <label for="id">ID do Cliente:</label>
```

```
        <input type="number" id="id" name="id" required>
```

```
        <button type="submit">Buscar</button>
```

```
    </form>
```

```
<?php else: ?>
```


Arquivo atualizarCliente.php

```
<!-- Se um cliente foi encontrado, exibe o formulário preenchido -->  
<form action="processarAtualizacao.php" method="POST">  
    <input type="hidden" name="id_cliente" value="<?=  
htmlspecialchars($cliente['id_cliente']) ?>">  
  
    <label for="nome">Nome:</label>  
    <input type="text" id="nome" name="nome" value="<?=  
htmlspecialchars($cliente['nome']) ?>" readonly  
    onclick="habilitarEdicao('nome')">
```

Arquivo atualizarCliente.php

```
<label for="endereco">Endereço:</label>  
<input type="text" id="endereco" name="endereco"  
value="<?= htmlspecialchars($cliente['endereco']) ?>" readonly  
onclick="habilitarEdicao('endereco')">
```

```
<label for="telefone">Telefone:</label>  
<input type="text" id="telefone" name="telefone"  
value="<?= htmlspecialchars($cliente['telefone']) ?>" readonly  
onclick="habilitarEdicao('telefone')">
```

Arquivo atualizarCliente.php

```
<label for="email">E-mail:</label>
  <input type="email" id="email" name="email" value="<?=
htmlspecialchars($cliente['email']) ?>" readonly
onclick="habilitarEdicao('email')">

  <button type="submit">Atualizar Cliente</button>
</form>
<?php endif; ?>

</body>
</html>
```

Arquivo para **ATUALIZAR** (PDO) no Banco de Dados

BACK END

Arquivo processarAtualizacao.php

```
<?php  
require 'conexao.php';
```

filter_var → Garante que o dado informado tem um formato válido.

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {
```

```
    $conexao = conectarBanco();
```

FILTER_SANITIZE_NUMBER_INT → Evita inserção de caracteres inesperados.

```
    $id = filter_var($_POST["id_cliente"], FILTER_SANITIZE_NUMBER_INT);
```

```
    $nome = htmlspecialchars(trim($_POST["nome"]));
```

```
    $endereco = htmlspecialchars(trim($_POST["endereco"]));
```

```
    $telefone = htmlspecialchars(trim($_POST["telefone"]));
```

```
    $email = filter_var($_POST["email"], FILTER_VALIDATE_EMAIL);
```

```
    if (!$id || !$email) {
```

```
        die("Erro: ID inválido ou e-mail incorreto.");
```

```
    }
```


Arquivo processarAtualizacao.php

PAUSA PARA DETALHAR E EXPLICAR!

O QUE `FILTER_VAR()` FAZ?

`filter_var()` é uma função nativa do **PHP** usada para validar e sanitizar dados.

O primeiro parâmetro é o **valor** a ser analisado (`$_POST["email"]`), e o segundo é o **filtro** aplicado (`FILTER_VALIDATE_EMAIL`), que **verifica se a string tem um formato válido de e-mail**.

Arquivo `processarAtualizacao.php`

PAUSA PARA DETALHAR E EXPLICAR!

- O filtro aplicado (`FILTER_SANITIZE_NUMBER_INT`), **que remove todos os caracteres não numéricos**, permitindo apenas números positivos e negativos.
- **Evita entrada de caracteres inválidos** → Se o usuário **digitar 123abc**, ele **será convertido para 123**.
- **Garante integridade no banco de dados** → O ID precisa ser um número válido para funcionar corretamente nas operações de DELETE e UPDATE.

Arquivo processarAtualizacao.php

PAUSA PARA DETALHAR E EXPLICAR!

✓ Se o usuário inserir **456xyz**, a variável **\$id** será convertida para **456**.

✗ Se inserir **“texto”**, a variável retornará **vazia** ou **falsa**, indicando erro.

💡 **Conclusão:** Essa garante que somente valores numéricos sejam aceitos antes de executar operações no banco de dados.

Arquivo processarAtualizacao.php

PAUSA PARA DETALHAR E EXPLICAR!

✓ Por que validar e-mails antes de inserir no banco?

1 **Evita erros** → Se alguém digitar um e-mail inválido, como "abc@@email.com", essa função retornará false, impedindo que um dado incorreto seja salvo.

2 **Evita registros desnecessários** → Se não validarmos, poderíamos armazenar valores como "testeemail" que não são e-mails reais.

3 **Melhora a segurança** → Impede que usuários insiram dados maliciosos no campo de e-mail.

Arquivo `processarAtualizacao.php`

PAUSA PARA DETALHAR E EXPLICAR!

- ✓ Se o usuário digitar "**usuario@dominio.com**", a variável `$email` **conterá esse valor corretamente**.
- ✗ Se digitar "**usuario@dominio**", a função **retornará false**, indicando erro.

💡 **Conclusão:** Essa validação é uma boa prática e garante que apenas e-mails corretos sejam armazenados.

Arquivo processarAtualizacao.php

VAMOS CONTINUAR O CODIGO!

```
$sql = "UPDATE cliente SET nome = :nome, endereco = :endereco,  
telefone = :telefone, email = :email WHERE id_cliente = :id";
```

```
$stmt = $conexao->prepare($sql);  
$stmt->bindParam(":id", $id, PDO::PARAM_INT);  
$stmt->bindParam(":nome", $nome);  
$stmt->bindParam(":endereco", $endereco);  
$stmt->bindParam(":telefone", $telefone);  
$stmt->bindParam(":email", $email);
```

Arquivo processarAtualizacao.php

```
try {  
    $stmt->execute();  
    echo "Cliente atualizado com sucesso!";  
} catch (PDOException $e) {  
    error_log("Erro ao atualizar cliente: " . $e->getMessage());  
    echo "Erro ao atualizar registro."  
}  
}  
?>
```

Formulário para APAGAR (PDO) no Banco de Dados

FRONT END

Este arquivo terá um formulário **HTML** para que o usuário **insira o ID** do cliente a ser excluído.

O formulário **envia os dados via POST** para **processarDelecao.php**

Arquivo deletarCliente.php

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Excluir Cliente</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h2>Excluir Cliente</h2>
```

Arquivo deletarCliente.php

```
<form action="processarDelecao.php" method="POST">  
  <label for="id">ID do Cliente:</label>  
  <input type="number" id="id" name="id" required>  
  
  <button type="submit">Excluir Cliente</button>  
</form>  
</body>  
</html>
```

Arquivo para APAGAR (PDO) no Banco de Dados

BACK END

Arquivo processarDelecao.php

```
<?php
```

```
require_once 'conexao.php';
```

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    $conexao = conectarBanco();
```

```
    $id = filter_var($_POST["id"], FILTER_SANITIZE_NUMBER_INT);
```

```
    if (!$id) {  
        die("Erro: ID inválido.");  
    }
```


Arquivo processarDelecao.php

```
$sql = "DELETE FROM cliente WHERE id_cliente = :id";  
$stmt = $conexao->prepare($sql);  
$stmt->bindParam(":id", $id, PDO::PARAM_INT);
```

```
try {  
    $stmt->execute();  
    echo "Cliente excluído com sucesso!";  
} catch (PDOException $e) {  
    error_log("Erro ao excluir cliente: " . $e->getMessage());  
    echo "Erro ao excluir cliente.";
```

```
}
```

```
}
```

```
?>
```

Arquivo para PESQUISAR CLIENTE (PDO) no Banco de Dados

FRONT END e BACK END

Arquivo pesquisarCliente.php

```
<?php  
require_once 'conexao.php';  
  
$conexao = conectarBanco();  
  
$busca = $_GET['busca'] ?? '';
```

Arquivo pesquisarCliente.php

```
if (!$busca) {  
    ?>  
    <form action="pesquisarCliente.php" method="GET">  
        <label for="busca">Digite o ID ou Nome:</label>  
        <input type="text" id="busca" name="busca" required>  
        <button type="submit">Pesquisar</button>  
    </form>  
    <?php  
    exit;  
}
```

Arquivo pesquisarCliente.php

```
// Escolhe entre busca por ID ou Nome e faz a consulta diretamente
if (is_numeric($busca)) {
    $stmt = $conexao->prepare("SELECT id_cliente,
nome, endereco, telefone, email FROM cliente WHERE
id_cliente = :id");
    $stmt->bindParam(":id", $busca, PDO::PARAM_INT);
} else {
```


Arquivo pesquisarCliente.php

```
$stmt = $conexao->prepare("SELECT id_cliente, nome,  
endereco, telefone, email FROM cliente WHERE nome  
LIKE :nome");  
    $buscaNome = "%$busca%";  
    $stmt->bindParam(":nome", $buscaNome,  
PDO::PARAM_STR);  
}  
  
$stmt->execute();  
$clientes = $stmt->fetchAll()
```

Arquivo pesquisarCliente.php

```
if (!$clientes) {  
    die("Erro: Nenhum cliente encontrado.");  
}  
?>  
<table border="1">  
    <tr>  
        <th>ID</th>  
        <th>Nome</th>  
        <th>Endereço</th>  
        <th>Telefone</th>  
        <th>E-mail</th>  
        <th>Ação</th>  
    </tr>
```

Arquivo pesquisarCliente.php

```
<?php foreach ($clientes as $cliente): ?>
```

```
<tr>
```

```
<td><?= htmlspecialchars($cliente['id_cliente']) ?></td>
```

```
<td><?= htmlspecialchars($cliente['nome']) ?></td>
```

```
<td><?= htmlspecialchars($cliente['endereco']) ?></td>
```

```
<td><?= htmlspecialchars($cliente['telefone']) ?></td>
```

```
<td><?= htmlspecialchars($cliente['email']) ?></td>
```

```
<td>
```

Arquivo pesquisarCliente.php

```
<a href="atualizarCliente.php?id=<?= $cliente['id_cliente']  
?>">Editar</a>  
    </td>  
</tr>  
<?php endforeach; ?>  
</table>
```

Arquivo para **BUSCAR** **CLIENTE (PDO)** no Banco de Dados

FRONT END** e **BACK END

Arquivo buscarCliente.php

```
<?php
require_once 'conexao.php';

$conexao = conectarBanco();

// Consulta todos os clientes do banco
// Ordena por nome para melhor visualização
$sql = "SELECT id_cliente, nome, endereco, telefone, email
FROM cliente ORDER BY nome ASC";
$stmt = $conexao->prepare($sql);
$stmt->execute();
$clientes = $stmt->fetchAll();
?>
```

Arquivo buscarCliente.php

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Lista de Clientes</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <h2>Todos os Clientes Cadastrados</h2>
```

Arquivo buscarCliente.php

```
<?php if (!$clientes): ?>  
    <p style="color:red;">Nenhum cliente encontrado no banco  
de dados.</p>  
<?php else: ?>  
    <table border="1">  
        <tr>  
            <th>ID</th>  
            <th>Nome</th>  
            <th>Endereço</th>  
            <th>Telefone</th>  
            <th>E-mail</th>  
            <th>Ação</th>  
        </tr>
```

Arquivo buscarCliente.php

```
<?php foreach ($clientes as $cliente): ?>
```

```
<tr>
```

```
<td><?= htmlspecialchars($cliente['id_cliente']) ?></td>
```

```
<td><?= htmlspecialchars($cliente['nome']) ?></td>
```

```
<td><?= htmlspecialchars($cliente['endereco']) ?></td>
```

```
<td><?= htmlspecialchars($cliente['telefone']) ?></td>
```

```
<td><?= htmlspecialchars($cliente['email']) ?></td>
```

```
<td>
```

```
<a href="atualizarCliente.php?id=<?=
$cliente['id_cliente'] ?>">Editar</a>
```

```
</td>
```

```
</tr>
```

Arquivo buscarCliente.php

```
<?php endforeach; ?>
```

```
</table>
```

```
<?php endif; ?>
```

```
</body>
```

```
</html>
```


REQUIRE
Vs
REQUIRE_ONCE

The background of the image features a dark blue gradient with a subtle pattern of white stars and nebulae. Overlaid on this are several faint, light blue technical diagrams. In the top right, there is a large circular gauge with concentric rings and numerical markings from 0 to 210. In the bottom right, there is a diagram of two concentric circles with arrows indicating a clockwise flow. In the bottom left, there is a partial view of a circular diagram with an arrow pointing counter-clockwise.

Use **require_once** para segurança (evita erros de redefinição).

Use **require** em códigos pequenos onde a inclusão repetida não será um problema.

Use **require_once** quando:

- - Você quer garantir que o arquivo seja incluído **apenas uma vez**.
- - Está incluindo **arquivos com funções ou classes**, evitando erro de **"Cannot redeclare function"**.
- - Trabalha com **arquivos globais**, como configuração de banco de dados (**conexao.php**).

Use **require** quando:

- - Precisa incluir um arquivo **toda vez que ele for chamado**, sem restrição.
- - Está lidando com **scripts pequenos**, onde você sabe que **não há risco de duplicação**.

Imagine que **require** é como pegar um livro da biblioteca todas as vezes que precisar.

Já **require_once** é como pegar o livro e deixar ele sobre a mesa para consultar sempre que necessário, sem precisar ir buscar novamente.

The background is a dark blue gradient with a subtle pattern of white stars and nebulae. Overlaid on this are several faint, light blue technical diagrams. In the top right, there is a large circular gauge with concentric circles and radial tick marks, resembling a speedometer or a data visualization. In the bottom right, there is a diagram of a circular path with arrows indicating a clockwise direction. In the bottom left, there is a partial view of a similar circular diagram. The text is centered in the middle of the image.

Arquivo de FUNÇÕES no Banco de Dados

Arquivo `funcoesBanco.php`

O arquivo `funcoesBanco.php` organiza as funções de banco (**CRUD**), tornando o código limpo, reutilizável e seguro contra **SQL Injection**.

Desta forma você poderá entender como separar responsabilidades e como trabalhar com funções reutilizáveis no PDO.

Arquivo `funcoesBanco.php`

✓ Melhorias:

- Encapsula a lógica de inserção e listagem em funções reutilizáveis;
- Usa `prepare()` e `bindParam()` para evitar SQL Injection;
- `bindParam()` garante que os valores sejam tratados corretamente;
- `error_log()` registra erros no log sem expô-los ao usuário.

Arquivo funcoesBanco.php

```
<?php
```

```
require_once 'conexao.php';
```

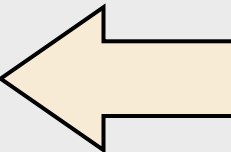
```
function obterClientes($conexao) {
```

```
    $stmt = $conexao->prepare("SELECT * FROM cliente");
```

```
    $stmt->execute();
```

```
    return $stmt->fetchAll();
```

```
}
```



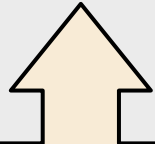
- Uso de **fetchAll()**: Retorna todos os registros de uma vez.

```
$conexao = conectarBanco();
```

```
$clientes = obterClientes($conexao);
```

Arquivo funcoesBanco.php

```
/*  
foreach ($clientes as $cliente) {  
    echo "Nome: " . htmlspecialchars($cliente["nome"]) . "<br  
/>";  
    echo "Telefone: " . htmlspecialchars($cliente["telefone"]) .  
    "<br />";  
}  
*/
```



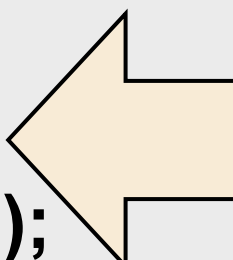
- Uso de **htmlspecialchars()**: Evita problemas com caracteres especiais na exibição.

O ***foreach*** que coloquei comentado aqui no código serve para exemplificar como exibir os clientes fora da tabela **HTML**, apenas com ***echo***.

Arquivo funcoesBanco.php

```
function inserirCliente($conexao, $nome, $endereco, $telefone,
$email, $idade) {
    $sql = "INSERT INTO cliente (nome, endereco, telefone,
email, idade)
        VALUES (:nome, :endereco, :telefone, :email, :idade)";
```

```
$stmt = $conexao->prepare($sql);
$stmt->bindParam(":nome", $nome);
$stmt->bindParam(":endereco", $endereco);
$stmt->bindParam(":telefone", $telefone);
$stmt->bindParam(":email", $email);
$stmt->bindParam(":idade", $idade, PDO::PARAM_INT);
```



- Uso de **prepare()** e **bindParam()**: Evita SQL Injection e melhora segurança.

Arquivo funcoesBanco.php

```
try {  
    $stmt->execute();  
    echo "Registro inserido com sucesso!";  
} catch (PDOException $e) {  
    error_log("Erro ao inserir cliente: " . $e->getMessage());  
    echo "Erro ao inserir registro.";
```

- Tratamento de erro com **try-catch**: Evita exposição de informações ao usuário.

```
}  
}  
?>
```

Arquivo funcoesBanco.php

```
function deletarCliente($id) {  
    $conexao = conectarBanco();  
  
    $sql = "DELETE FROM cliente WHERE id_cliente = :id";  
  
    $stmt = $conexao->prepare($sql);  
    $stmt->bindParam(":id", $id, PDO::PARAM_INT);
```

CONTINUA...

Arquivo funcoesBanco.php

...CONTINUAÇÃO

```
try {  
    $stmt->execute();  
    return true;  
} catch (PDOException $e) {  
    error_log("Erro ao excluir cliente: " . $e->getMessage());  
    return false;  
}  
}
```

CONTINUA...

Arquivo funcoesBanco.php

...CONTINUAÇÃO

```
function atualizarCliente($id, $nome, $endereço, $telefone,
$email) {
    $conexao = conectarBanco();

    $sql = "UPDATE cliente SET nome = :nome, endereço =
:endereço, telefone = :telefone, email = :email WHERE
id_cliente = :id";
```

CONTINUA...

Arquivo funcoesBanco.php

...CONTINUAÇÃO

```
$stmt = $conexao->prepare($sql);  
$stmt->bindParam(":id", $id, PDO::PARAM_INT);  
$stmt->bindParam(":nome", $nome);  
$stmt->bindParam(":endereco", $endereco);  
$stmt->bindParam(":telefone", $telefone);  
$stmt->bindParam(":email", $email);
```

CONTINUA...

Arquivo funcoesBanco.php

...CONTINUAÇÃO

```
try {  
    $stmt->execute();  
    return true;  
} catch (PDOException $e) {  
    error_log("Erro ao atualizar cliente: " . $e->getMessage());  
    return false;  
}  
}
```

CONTINUA...

Arquivo funcoesBanco.php

...CONTINUAÇÃO

```
function listarClientes() {  
    $conexao = conectarBanco();  
    $stmt = $conexao->query("SELECT id_cliente, nome,  
endereco, telefone, email FROM cliente ORDER BY nome  
ASC");  
    return $stmt->fetchAll();  
}
```

CONTINUA...

Arquivo funcoesBanco.php

...CONTINUAÇÃO

```
function buscarClientePorId($idCliente) {  
    $conexao = conectarBanco();  
    $stmt = $conexao->prepare("SELECT id_cliente, nome,  
endereco, telefone, email FROM cliente WHERE id_cliente =  
:id");  
    $stmt->bindParam(":id", $idCliente, PDO::PARAM_INT);  
    $stmt->execute();  
    return $stmt->fetch();  
}
```

CONTINUA...

Arquivo funcoesBanco.php

...CONTINUAÇÃO

```
function buscarClientesPorNome($nome) {  
    $conexao = conectarBanco();  
    $stmt = $conexao->prepare("SELECT id_cliente, nome,  
endereco, telefone, email FROM cliente WHERE nome LIKE  
:nome");  
    $nomeBusca = "%$nome%";  
    $stmt->bindParam(":nome", $nomeBusca,  
PDO::PARAM_STR);  
    $stmt->execute();  
    return $stmt->fetchAll();  
}  
?>
```

FIM!

FIM!